

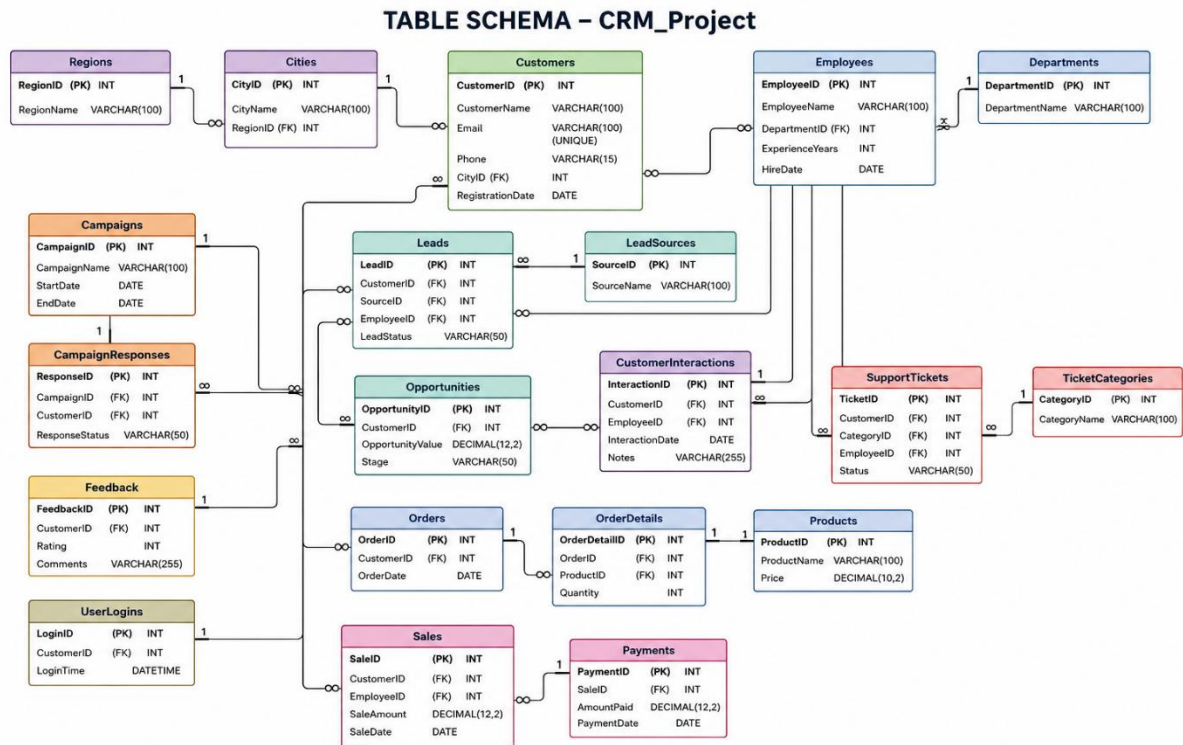
CUSTOMER RELATIONSHIP MANAGEMENT(CRM)

NAME: K. SAI DEEPTHI

ID:146860

GITHUB: <https://github.com/kommurusaideepthi25697-tech/CRM-Database-Management-System.git>

Table



CUSTOMER RELATIONSHIP MANAGEMENT(CRM)

BASIC QUERIES

1. Find all customers from Chennai.
2. Retrieve all open support tickets.
3. Find total customers in each city.
4. Display employees working in Sales department.
5. Find total sales handled by each employee.
6. Retrieve customers without any sales.
7. Find all sales above ₹50,000.
8. Display top 10 highest sales amounts.
9. Find average customer rating.
10. Retrieve customers registered this month.
11. Find customers who made more than one purchase.
12. Retrieve employees with more than 5 years experience.
13. Find total tickets raised by each customer.
14. Display all unresolved tickets.
15. Find customers who never provided feedback.
16. Retrieve cities having more than 100 customers.
17. Display feedback ratings below 3.
18. Find total CRM revenue generated.
19. Retrieve sales count month-wise.
20. Find total opportunities in each stage.

CUSTOMER RELATIONSHIP MANAGEMENT(CRM)

INTERMEDIATE

1. Find the top 5 employees who generated the highest sales revenue.
2. Retrieve customers who have multiple business opportunities in the CRM system.
3. Identify the city that generated the maximum revenue.
4. Display customers who have raised more than 3 support tickets.
5. Find the most common lead source that brings customers to the company.
6. Retrieve employees whose average customer feedback rating is greater than 4.5.
7. Find the customer with the second-highest revenue contribution.
8. Identify employees who have closed the maximum number of deals.
9. Calculate the average sale amount generated from each city.
10. Display the top 10 customers based on total revenue generated.
11. Find customers who were created as leads but never converted into sales.
12. Identify duplicate customer records based on email addresses.
13. Calculate the percentage contribution of each employee to the total company revenue.
14. Calculate the percentage contribution of each city to the total company revenue.
15. Rank employees based on their sales performance and revenue generation.
16. Find the total revenue generated by each department.
17. Identify customers who purchased the most expensive product.
18. Find the marketing campaign that received the highest customer response.
19. Analyze monthly revenue trends generated by sales activities.
20. Find customers who have purchased more than one product from the company.

CUSTOMER RELATIONSHIP MANAGEMENT(CRM)

ADVANCED SQL (TRIGGERS & PROCEDURES)

1. Revenue Status Automation Trigger
2. Ticket Assignment Procedure
3. Duplicate Customer Prevention Procedure
4. UPDATE SALES INFORMATION WITHOUT DUPLICATE SALE ID

Revenue Status Automation Trigger

```
DELIMITER //  
  
CREATE TRIGGER RevenueAutomation  
AFTER INSERT ON Sales  
FOR EACH ROW  
BEGIN  
    UPDATE Opportunities  
    SET Stage='Closed Won'  
    WHERE CustomerID=NEW.CustomerID;  
END //  
  
DELIMITER ;
```

Ticket Assignment Procedure

```
DELIMITER //  
  
CREATE PROCEDURE AssignTicket(  
IN p_TicketID INT,  
IN p_EmployeeID INT  
)  
BEGIN  
  
    UPDATE SupportTickets  
    SET Status='Assigned'  
    WHERE TicketID=p_TicketID;  
  
END //  
  
DELIMITER ;
```

CUSTOMER RELATIONSHIP MANAGEMENT(CRM)

Duplicate Customer Prevention Procedure

```
DELIMITER //  
  
CREATE PROCEDURE AddCustomer(  
  IN p_CustomerID INT,  
  IN p_Name VARCHAR(100),  
  IN p_Email VARCHAR(100)  
)  
BEGIN  
  
  DECLARE CustomerCount INT;  
  
  SELECT COUNT(*)  
  INTO CustomerCount  
  FROM Customers  
  WHERE Email=p_Email;  
  
  IF CustomerCount > 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT='Duplicate Customer';  
  
  ELSE  
  
    INSERT INTO Customers  
    VALUES(p_CustomerID,p_Name,p_Email,NULL,NULL,NULL,CURDATE());  
  END IF;  
  
END //  
  
DELIMITER ;
```

CUSTOMER RELATIONSHIP MANAGEMENT(CRM)

UPDATE SALES INFORMATION WITHOUT DUPLICATE SALE ID

```
DELIMITER//
```

```
CREATE PROCEDURE `SALESINFO`(IN P_SALEID INT,IN P_CUSTOMERID INT,IN  
P_EMPLOYEEID INT,IN P_SALEAMOUNT INT)
```

```
BEGIN
```

```
DECLARE NEWSALEID INT;
```

```
SELECT COUNT(*)
```

```
INTO NEWSALEID FROM SALES
```

```
WHERE SALEID=P_SALEID;
```

```
IF NEWSALEID>0
```

```
THEN
```

```
SIGNAL SQLSTATE'45000'
```

```
SET MESSAGE_TEXT='DUPLICATE_SALE';
```

```
ELSE
```

```
INSERT INTO SALES
```

```
VALUES(P_SALEID,P_CUSTOMERID,P_EMPLOYEEID,P_SALEAMOUNT,CURDATE());
```

```
END IF;
```

```
END
```